BANQUE AFRICAINE DE DÉVELOPPEMENT · DATA INNOVATION FOR AFRICA (DI4A)

Passer à l'échelle

Technologies big data pour les systèmes statistiques

Des fichiers bruts aux indicateurs de confiance : les six briques de toute plateforme de données



Banque africaine de développement – Data Innovation for Africa (DI4A)

Pourquoi les INS ont besoin de moteurs

≈ 611 m

Tuiles Ookla Speedtest

Tuiles de niveau 16, réseaux fixe et mobile, chaque trimestre depuis 2019

≈ 3 milliards

Grille WorldPop à 100 m

Calcul : ≈ 30 millions de km² × 100 cellules par km², par couche et par an

Quotidien

Radiance Black Marble

Lumières nocturnes VNP46A2 à ~500 m (15 secondes d'arc)

Le mur de la mémoire — et trois façons de le franchir

Tient aisément en mémoire vive

pandas, R — règle empirique : RAM ≈ 5–10× la taille des données

Tient sur le disque d'une machine

DuckDB, Polars — lecture des colonnes Parquet, hors mémoire

Dépasse une machine ou doit être partagé

Stockage objet, Spark, lakehouse

Doit répondre vite à des requêtes interactives

Moteur de recherche : Elasticsearch

Sources : Ookla Open Data (GitHub) ; WorldPop ; NASA Black Marble ; W. McKinney, « Apache Arrow and the 10 things I hate about pandas » (2017).

Les six briques de toute plateforme de données

01 Ingestion
Faire entrer les données, intactes et documentées

02 Stockage
Les conserver à bas coût, en sécurité, lisibles vite

03 Traitement distribué
Nettoyer, apparier et calculer en parallèle

04 Indexation et recherche
Retrouver et interroger en millisecondes

05 Orchestration
Planifie chaque étape, gère les dépendances, relance les échecs, alerte

06 Métadonnées et versionnage
Décrit chaque jeu de données, trace chaque chiffre, versionne code, données et environnement

Transversales : elles couvrent les quatre briques ci-dessus

SECTION 01 SUR 06

01

Ingestion

Comment les données entrent-elles dans le système — et restent-elles fiables dès le premier octet ?



Cinq façons de faire entrer les données

Transfert par lots

Fichiers complets selon un calendrier

EXEMPLE

Tuiles Ookla : Parquet trimestriel depuis un bucket S3 public

Heures–jours

Appel d'API

Requêtes programmées vers un fournisseur

EXEMPLE

Lumières nocturnes via NASA Earthdata ou Google Earth Engine

Minutes–jours

Capture des changements

Seules les lignes nouvelles ou modifiées sont copiées

EXEMPLE

Registres fiscaux, d'entreprises et d'état civil

Minutes

Streaming

Événements en flux continu dans un journal (Kafka)

EXEMPLE

Collecte de prix en ligne, capteurs, événements réseau

Secondes

Dépôt sécurisé

Transfert chiffré et journalisé

EXEMPLE

Microdonnées partenaires, ex. extraits d'opérateurs télécoms

Selon l'accord

Délais indicatifs. Bucket Ookla : s3://ookla-open-data (AWS Open Data Registry).

Le contrat d'ingestion : cinq règles à l'entrée

1. **Ne jamais modifier les données brutes**
   Conserver les octets tels que reçus ; les corrections se font en aval.

2. **Tracer la provenance**
   Source, URL, date et licence (Ookla : CC BY-NC-SA 4.0).

3. **Vérifier l'intégrité**
   Sommes de contrôle, nombre de lignes et de fichiers attendus par période.

4. **Contrôler le schéma**
   Ookla a ajouté des colonnes en 2022–23 : détecter la dérive, jamais en silence.

5. **Rendre les chargements idempotents**
   Relancer un même trimestre donne le même résultat, sans doublons.

ENREGISTREMENT DE PROVENANCE (ILLUSTRATIF)

```
# provenance.json — un par fichier brut
{
  "source": "Speedtest by Ookla",
  "layer": "mobile",
  "period": "2025-Q4",
  "uri": "s3://ookla-open-data/
    parquet/performance/type=mobile/
    year=2025/quarter=4/",
  "retrieved_at": "2026-01-15T08:12Z",
  "licence": "CC BY-NC-SA 4.0",
  "sha256": "9f2c…e41a",
  "schema_version": "2023-Q3",
  "loaded_by": "pipeline v1.4.0"
}
```

Source : documentation Ookla Open Data (historique des colonnes et licence).

SECTION 02 SUR 06

02

Stockage

Où conserver les données pour qu'elles soient peu coûteuses, rapides à lire et impossibles à perdre ?



Stockage distribué : découper et répliquer

Partitionner

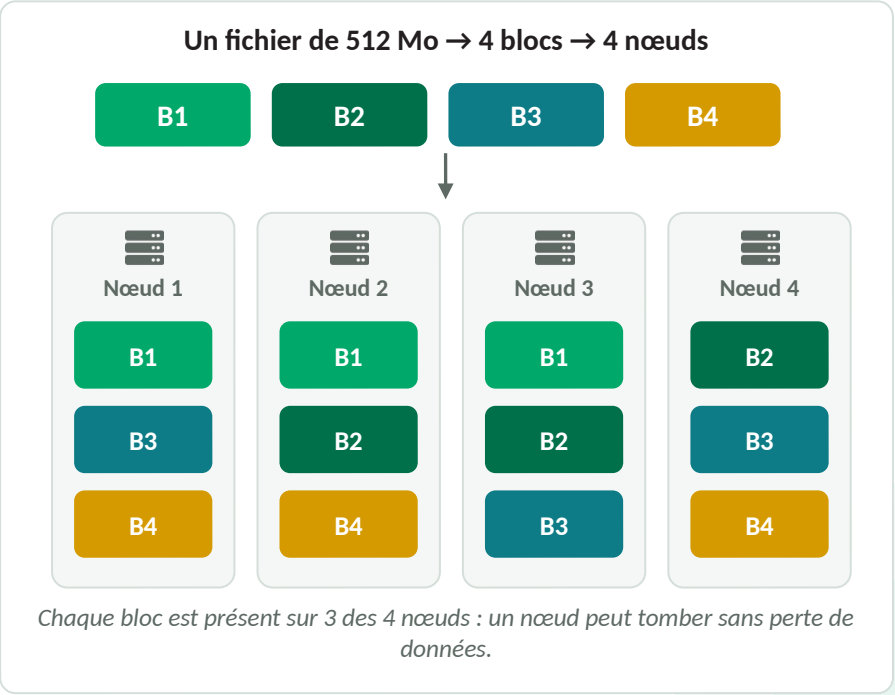
Les gros fichiers sont découpés en blocs de taille fixe — 128 Mo par défaut dans HDFS — répartis sur les nœuds.

Répliquer

Chaque bloc est stocké sur trois nœuds par défaut. La panne matérielle est considérée comme normale.

Calculer sur place

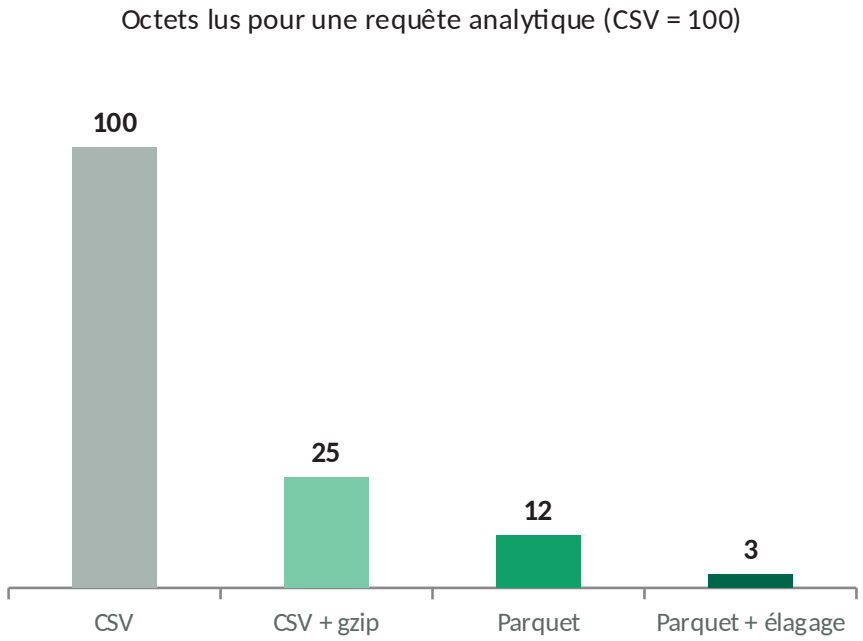
Envoyer le code vers les données, pas l'inverse : la clé de la vitesse à grande échelle.



Chaque bloc est présent sur 3 des 4 nœuds : un nœud peut tomber sans perte de données.

Source : guide Apache Hadoop « HDFS Architecture » (taille de bloc et facteur de réplication par défaut).

Le format de fichier est un choix de performance



Lire des colonnes, pas des lignes

Une vitesse moyenne pondérée n'utilise que 2 des 11 colonnes des fichiers Ookla récents.

Compresser les valeurs proches

Dans une colonne, les valeurs voisines se ressemblent : la compression est bien meilleure.

Ignorer l'inutile

Les min/max par groupe de lignes permettent de sauter des blocs entiers (predicate pushdown).

Ordres de grandeur illustratifs, pas un banc d'essai. Sources : documentation Apache Parquet ; schéma Ookla Open Data.

Entrepôt, lac, lakehouse

Entrepôt de données

CONTENU
Tables structurées et curées

SCHÉMA
Schéma imposé à l'écriture

ATOUT
SQL rapide, gouvernance forte

POINT DE VIGILANCE
Rigide ; coûteux à faire grandir

EXEMPLES
BigQuery, Snowflake, PostgreSQL

Lac de données

CONTENU
Fichiers bruts, tout format

SCHÉMA
Schéma interprété à la lecture

ATOUT
Bon marché, souple, illimité

POINT DE VIGILANCE
Devient un « marécage » sans discipline

EXEMPLES
Stockage objet + Parquet, GeoTIFF, HDF5

Lakehouse

CONTENU
Du brut au curé, en un seul lieu

SCHÉMA
À la lecture + tables contrôlées

ATOUT
Transactions, voyage dans le temps, évolution du schéma

POINT DE VIGILANCE
Exige une maturité en ingénierie

EXEMPLES
Apache Iceberg, Delta Lake, Apache Hudi

Source : Armbrust et al., « Lakehouse: A New Generation of Open Platforms… », CIDR 2021.

Le modèle médaillon appliqué à nos données

BRONZE

Brut, tel que reçu

- Fichiers Parquet trimestriels Ookla
- Rasters GeoTIFF WorldPop
- Granules HDF5 Black Marble
- Fichiers de limites administratives

ARGENT

Validé et relié

- Tuiles découpées à l'emprise nationale
- Chaque tuile rattachée à son unité administrative
- Population par tuile issue de WorldPop
- Radiance avec masques nuages et qualité

OR

Produits statistiques

- Connectivité pondérée par la population, par région et trimestre
- Statistiques zonales des lumières nocturnes par niveau administratif
- Tableau de bord et tables sur GitHub

Contrôle qualité à chaque flèche : chaque zone a un responsable, un contrat de schéma et des tests. Rien n'avance tant qu'ils ne sont pas validés.

Ce qu’apporte un format de table lakehouse

Transactions ACID

Des écritures simultanées ne laissent jamais une table incohérente.

Voyage dans le temps

Interroger une table telle qu’elle était à une date passée.

Évolution du schéma

Ajouter une colonne sans réécrire l’historique.

Élagage des partitions

Une requête sur un trimestre ignore tous les autres.

SPARK SQL + APACHE ICEBERG

```
-- La table exactement telle qu’elle
-- était lors de la publication du T2
SELECT admin1, dl_kbps_weighted
FROM   lake.gold.connectivity
TIMESTAMP AS OF '2026-07-15 00:00:00'
WHERE  year = 2026 AND quarter = 2;
```

Pourquoi c’est utile au statisticien

Une révision s’explique en rejouant l’état exact des entrées d’un chiffre publié.

Sources : documentation Apache Iceberg et Delta Lake. Noms de tables et de colonnes illustratifs.

SECTION 03 SUR 06

03

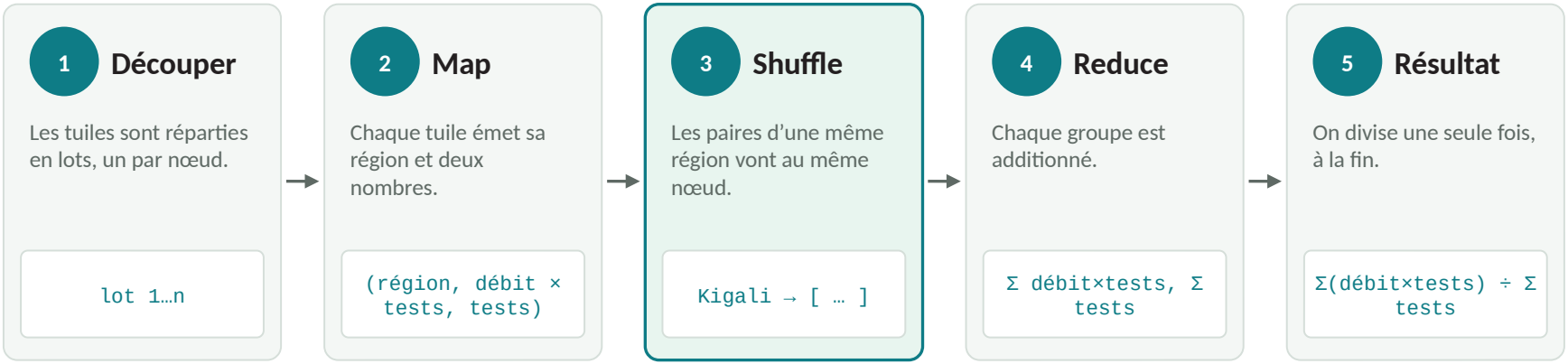
Traitement distribué

Comment cent machines calculent-elles une statistique comme si elles n'en faisaient qu'une ?



MapReduce : l'idée au cœur de chaque moteur

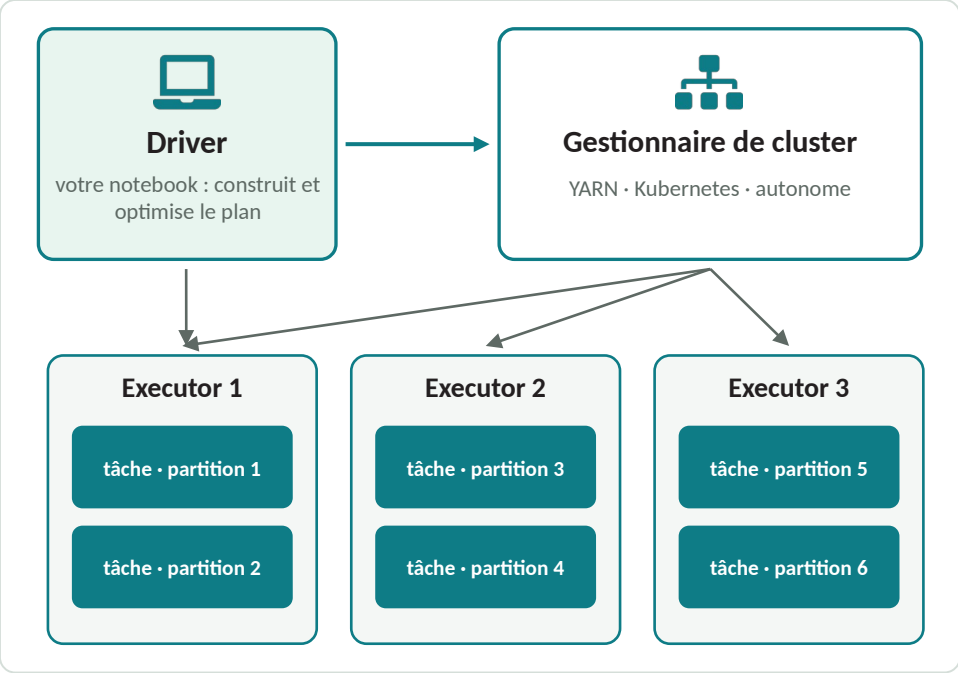
Exemple — débit descendant moyen pondéré par les tests, par région, à partir des tuiles Ookla



Shuffle = l'étape réseau coûteuse

Ne jamais faire la moyenne des moyennes. Chaque tuile Ookla est déjà une moyenne sur un nombre de tests différent. Transportez sommes et effectifs et divisez à la fin : c'est correct et parfaitement parallèle. Médianes et quantiles ne se décomposent pas ainsi : les moteurs les approchent.

Apache Spark : comment un traitement s'exécute



Partitions

Les données sont découpées ; une tâche traite une partition.

Évaluation paresseuse

Les transformations sont enregistrées ; une action (write, count) déclenche le calcul.

Optimiseur (Catalyst)

Réécrit tout le plan : filtres au plus tôt, choix des jointures.

Shuffle

Regroupements et jointures déplacent les données sur le réseau : le goulot habituel.

Sources : documentation Apache Spark (Cluster Mode Overview) ; Zaharia et al., Communications of the ACM, 2016.

Spark en pratique : un indicateur de connectivité

PYSPARK

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("connectivity").getOrCreate()

# 1. Lire les tuiles validées (argent), déjà rattachées aux régions
tiles = spark.read.parquet("s3://nso-lake/silver/ookla_admin/")

# 2. Débit moyen pondéré par les tests + diagnostics de couverture
kpi = (tiles
  .filter(F.col("tests") > 0)
  .groupBy("country", "admin1", "year", "quarter")
  .agg((F.sum(F.col("avg_d_kbps") * F.col("tests"))
        / F.sum("tests")).alias("dl_kbps_weighted"),
       F.sum("tests").alias("n_tests"),
       F.countDistinct("quadkey").alias("n_tiles")))

# 3. Publier en zone or, un dossier par trimestre
kpi.write.mode("overwrite") \
   .partitionBy("year", "quarter") \
   .parquet("s3://nso-lake/gold/connectivity/")
```

Identique à toute échelle

Même script sur un échantillon (mode local) ou sur tout un historique continental. Seule la taille du cluster change.

La statistique d'abord

n_tests et n_tiles accompagnent l'estimation : les chiffres régionaux trop fragiles sont signalés avant publication.

Une grammaire familière

Filtrer, regrouper, agréger, écrire : les mêmes verbes qu'en SQL, R dplyr ou pandas.

Noms de colonnes conformes au schéma Ookla Open Data ; chemins et noms de tables illustratifs.

Lots ou flux : choisir le modèle temporel

Par lots (batch)

Données bornées, traitées selon un calendrier

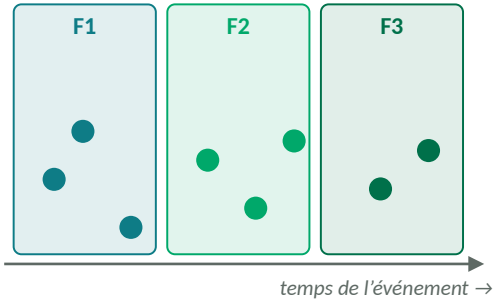
- Délai : heures à jours
- Simple à comprendre et à relancer
- Adapté aux statistiques mensuelles, trimestrielles et annuelles
- Outils : Spark, DuckDB, dbt, entrepôts SQL

En flux (streaming)

Données sans fin, traitées en continu

- Délai : secondes
- Fenêtres, watermarks et temps d'événement
- Adapté au nowcasting, au suivi, aux prix en ligne
- Outils : Apache Kafka, Apache Flink, Spark Structured Streaming

Fenêtres fixes (5 min)



Chaque événement appartient à une seule fenêtre ; chaque fenêtre produit un agrégat.

La seule question à se poser : une décision change-t-elle si le chiffre arrive une heure plus tard ? Sinon, choisissez les lots : moins cher, plus simple, plus facile à auditer. Les indicateurs Ookla trimestriels et les composites mensuels de lumières nocturnes relèvent des lots.

Dimensionner juste : la renaissance du nœud unique

Portable, en mémoire
Jusqu'à quelques Go · pandas · R data.frame

Nœud unique, en colonnes
Jusqu'à des centaines de Go · DuckDB · Polars · Apache Arrow

Cluster ou entrepôt cloud
Téraoctets, nombreux utilisateurs · Spark · Trino · BigQuery

Plateforme de streaming
Flux continus · Kafka + Flink

DUCKDB SQL

```
-- DuckDB : interroger Ookla sur place
INSTALL httpfs; LOAD httpfs;
SET s3_region = 'us-west-2';
SELECT count(*) AS tiles,
       sum(tests) AS tests
FROM read_parquet('s3://ookla-open-data/
  parquet/performance/type=mobile/
  year=2025/quarter=4/*.parquet');
```

Seules les colonnes utiles à la requête sont lues à travers le réseau.

L'erreur la plus coûteuse : exploiter un cluster pour des données qui tiennent sur une seule machine. Mesurez d'abord, puis passez à l'échelle.

Seuils empiriques. Sources : documentation DuckDB (httpfs, read_parquet) ; région du bucket Ookla selon l'AWS Open Data Registry.

04

Indexation et recherche

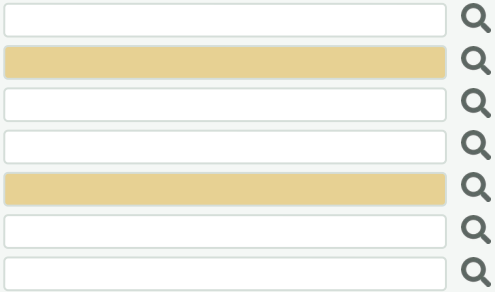
Pourquoi un moteur de recherche répond-il en millisecondes quand un parcours complet prend des minutes ?



Du parcours complet à l'indexation

Parcours complet

Lire chaque enregistrement, tester chacun



Le travail croît avec le volume : 10× plus de tuiles, ~10× plus de temps.

Index inversé

Chercher la valeur, aller droit aux enregistrements

`admin1 = "Kigali"`	→ doc 2, 5, 9, 14…
`admin1 = "Huye"`	→ doc 1, 7, 11…
`quarter = 2025-Q4`	→ doc 5, 7, 9…

Arbre B

Bases relationnelles : intervalles et tris

Index inversé

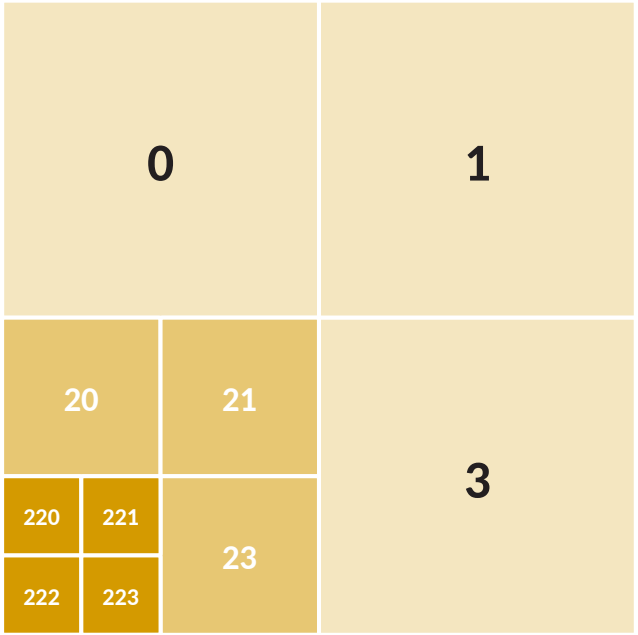
Moteurs de recherche : termes, mots-clés, filtres

Index spatial

Quadkey, geohash, arbre BKD : « à proximité / à l'intérieur »

Un index échange du travail à l'écriture et de l'espace disque contre de la vitesse à la lecture. Identifiants illustratifs.

Le quadkey : un index spatial déjà entre vos mains



Chaque niveau ajoute un chiffre (0–3)

16 chiffres

Un quadkey de niveau 16 ≈ une tuile Ookla de 611 m

Préfixe = voisinage

Garder les 12 premiers chiffres agrège en cellules d'≈ 9,8 km, sans fonction SIG

Identifiant stable

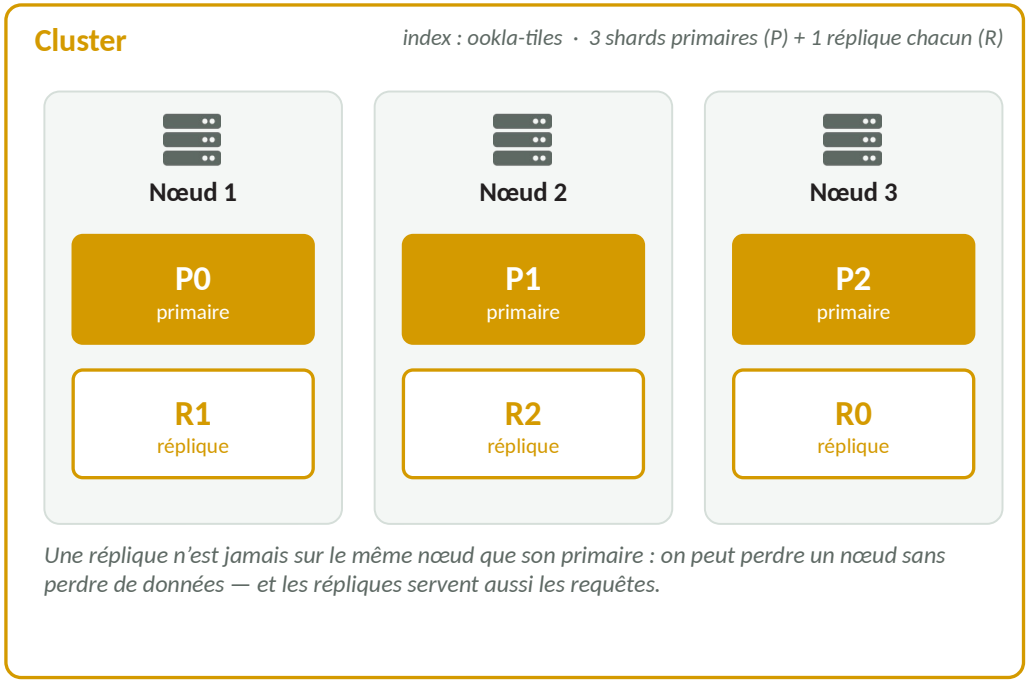
Relier la même tuile d'un trimestre à l'autre (de 2019 à aujourd'hui) par le seul quadkey

Formes réelles → geo_shape

Tuile face à une limite de district : Elasticsearch indexe les formes dans un arbre BKD

Sources : README Ookla Open Data ; système de tuiles Bing Maps ; documentation geo_shape d'Elasticsearch. Largeur = 40 075 km ÷ 2^zoom.

Elasticsearch en une image



Si vous pensez en SQL…

Table	→	**Index**
Ligne	→	**Document (JSON)**
Schéma	→	**Mapping**
Type de colonne	→	**Type de champ : keyword, integer, geo_shape…**
WHERE	→	**Requête (bool, term, range, geo_shape)**
GROUP BY	→	**Agrégation (terms, weighted_avg…)**

Source : Elasticsearch Reference (clusters, index, shards et mappings).

Mapping et requête : ce que vous taperez en labo

1 · MAPPING — LE SCHÉMA

```
PUT ookla-tiles
{ "mappings": { "properties": {
  "quadkey":    { "type": "keyword" },
  "admin1":     { "type": "keyword" },
  "quarter":    { "type": "date" },
  "avg_d_kbps": { "type": "integer" },
  "tests":      { "type": "integer" },
  "devices":    { "type": "integer" },
  "tile":       { "type": "geo_shape" }
}}}

// keyword   = valeur exacte
//             (filtrer, regrouper)
// geo_shape = vrai polygone de tuile
```

2 · REQUÊTE + AGRÉGATION — PAR RÉGION

```
GET ookla-tiles/_search
{ "size": 0,
  "query": { "geo_shape": { "tile": {
    "shape": { "type": "envelope",
      "coordinates": [[28.8, -1.0], [30.9, -2.9]] },
    "relation": "intersects" }}},
  "aggs": { "by_region": {
    "terms": { "field": "admin1" },
    "aggs": { "dl_weighted": { "weighted_avg": {
      "value":  { "field": "avg_d_kbps" },
      "weight": { "field": "tests" } }}}}}}

// size 0 = ne renvoyer que l'agrégation
// envelope = [haut-gauche], [bas-droit], lon d'abord
```

Sources : Elasticsearch Reference (requête geo_shape, agrégation weighted_avg). Rectangle approximatif du Rwanda ; le mapping du cluster de labo peut différer.

Un moteur de recherche : quand est-ce utile ?

	pandas	DuckDB sur Parquet	Elasticsearch
Idéal pour	Un analyste, travail exploratoire	SQL analytique lourd sur une machine	Nombreux utilisateurs, filtres et cartes interactifs
Mémoire	Tout le jeu de données en RAM	Lit les colonnes ; déborde sur disque	Répartie entre les nœuds
Géo et texte	Via GeoPandas, en mémoire	Extension spatiale	geo_shape et plein texte natifs
Latence interactive	Correcte tant que les données sont petites	Rapide pour parcours et jointures	Conçu pour des réponses < 1 s
Coût d'exploitation	Nul	Très faible	Un cluster à gérer, sécuriser et surveiller

Un moteur de recherche est un index, pas un système de référence. Le lakehouse reste la source de vérité ; l'index se reconstruit à partir de lui.

Comparaison qualitative — la partie 2 du labo vous demande de mesurer vous-mêmes temps de requête et empreinte mémoire.

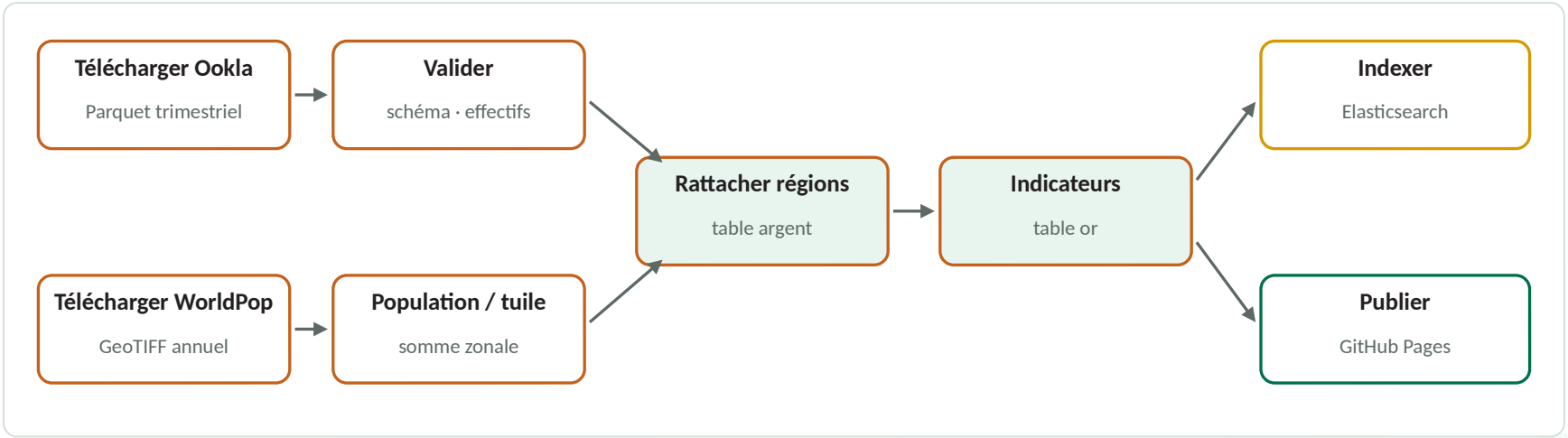
05

Orchestration

Qui lance la chaîne à 2 heures du matin — et que se passe-t-il quand une étape échoue ?



Une chaîne de traitement est un graphe de tâches



Planification · Dépendances · Relances · Alertes · Rattrapage

Outils, du plus simple au plus complet : cron · GitHub Actions → Apache Airflow · Dagster · Prefect · dbt pour les transformations SQL

La chaîne en code, avec contrôles qualité

APACHE AIRFLOW (API TASKFLOW)

```
from airflow.decorators import dag
import pendulum
# download_tiles, validate… = fonctions @task

@dag(schedule="@quarterly",
     start_date=pendulum.datetime(2019, 1, 1),
     catchup=True,            # = rattrapage
     default_args={"retries": 2})
def ookla_connectivity():
    raw    = download_tiles()
    ok     = validate(raw)       # contrôle
    pop    = population_per_tile()
    silver = join_admin(ok, pop)
    gold   = indicators(silver)
    index_elasticsearch(gold)
    publish_github(gold)

ookla_connectivity()
```

Contrôles qualité, écrits comme des tests

- ✓ `tests > 0 pour chaque tuile`
- ✓ `quadkey unique par trimestre`
- ✓ `avg_d_kbps dans une plage plausible`
- ✓ `chaque tuile dans une seule unité admin.`
- ✓ `nb de tuiles à ±X % du trimestre précédent`

En cas d'échec : arrêter, alerter une personne, ne jamais publier en silence. Outils : Great Expectations · tests dbt · Soda · pandera

Source : documentation Apache Airflow (API TaskFlow, catchup). Les seuils sont fixés par le méthodologue.

SECTION 06 SUR 06

06

Métadonnées et versionnage

Pouvez-vous prouver, des mois plus tard, comment un chiffre publié a été produit ?



Trois couches de métadonnées et le lignage

Techniques

Schéma, types, formats, partitions, volume, emplacement

Schéma Parquet · catalogue Iceberg

Statistiques

Concepts, définitions, nomenclatures, méthodes, limites

SDMX · DDI · GSIM

Opérationnelles

Lignage, journaux d'exécution, résultats qualité, accès

Journaux de chaîne · lignage du catalogue

Lignage : chaque chiffre publié remonte à des entrées exactes



Outils de catalogue : DataHub, OpenMetadata, CKAN ; description des jeux de données : W3C DCAT. Identifiants illustratifs.

Versionner trois choses, pas une seule

Code

- Dépôt Git, demandes de fusion relues
- Versions étiquetées (v1.2.0)
- Tests automatiques en CI

Données

- Instantanés de tables : Iceberg, Delta Lake
- Versionnage de fichiers : DVC, lakeFS
- La zone bronze n'est jamais écrasée

Environnement

- Dépendances figées : fichiers de verrouillage
- Environnements Conda ou conteneurs
- Mêmes entrées + même code + même env. = même chiffre

La version publiée = ce que produiront les jours 3 à 5

Version GitHub étiquetée + README (méthode, sources, licence, limites) + DOI citable via Zenodo + responsable de maintenance désigné

Sources : documentation de l'intégration GitHub–Zenodo ; documentation DVC et lakeFS.

Une plateforme qu'un INS peut faire grandir

DÉPART	Un serveur ou un portable
Stockage	Dossiers ou MinIO + Parquet
Traitement	DuckDB · Polars · pandas
Recherche	Facultative
Orchestration	cron · GitHub Actions
Métadonnées	README + fichiers de provenance
Versionnage	Git + GitHub

grandir selon les mesures →

ÉCHELLE	Plateforme nationale ou régionale
Stockage	Stockage objet + Iceberg / Delta
Traitement	Spark · Trino
Recherche	Cluster Elasticsearch
Orchestration	Airflow · Dagster
Métadonnées	DataHub / OpenMetadata, SDMX
Versionnage	Instantanés, conteneurs, CI/CD

Mutualiser : des hubs régionaux big data et des plateformes nationales partagées permettent de partager plateformes, code et compétences.

Choisir le bon moteur

Si votre situation est…	Privilégier	Éviter
Données en mémoire, un analyste	**pandas, R, DuckDB**	Monter un cluster
Dizaines à centaines de Go, SQL complexe	**DuckDB ou Polars sur un serveur puissant**	Copier des CSV entre portables
Téraoctets, plusieurs équipes, tables partagées	**Lakehouse + Spark ou Trino**	Partages réseau non gouvernés
Nombreux utilisateurs filtrant par lieu et période	**Elasticsearch alimenté par le lakehouse**	L'index comme unique copie
Latence inférieure à la minute réellement requise	**Kafka + Flink**	Du streaming pour des statistiques trimestrielles
Microdonnées confidentielles (CDR, fiscalité)	**Enclave sur site ou souveraine**	Cloud public sans base légale

Choisissez pour les données que vous avez, pas pour celles que vous imaginez avoir dans cinq ans.

Six choses à retenir



01

Ingérer avec rigueur

Garder les données brutes intactes ; tracer source, date et licence dès le premier octet.



02

Stocker en zones et en colonnes

Parquet + bronze / argent / or : vitesse et chaîne de production claire.



03

Calculer avec des sommes

Les moteurs parallèles exigent des statistiques décomposables ; dimensionner avant de multiplier.



04

Indexer pour l'interactivité

Un moteur de recherche répond vite, mais reste un index, jamais la source de vérité.



05

Automatiser avec des contrôles

Les chaînes s'exécutent au calendrier et s'arrêtent en cas d'échec : jamais de publication silencieuse.



06

Rendre chaque chiffre traçable

Versionner code, données et environnement ; documenter méthodes et limites.

QUESTIONS ET DISCUSSION

Merci



Sources et lectures complémentaires

Données utilisées dans les labos

- Ookla — Speedtest Global Performance Open Data, github.com/teamookla/ookla-open-data
- WorldPop — données démographiques spatiales ouvertes, worldpop.org
- NASA — lumières nocturnes Black Marble (produits VNP46)

Fondements

- Dean & Ghemawat, « MapReduce », OSDI 2004
- Zaharia et al., « Apache Spark: A Unified Engine for Big Data Processing », CACM 2016
- Armbrust et al., « Lakehouse », CIDR 2021
- McKinney, « Apache Arrow and the 10 Things I Hate About pandas », 2017

Documentation technique

- Apache Hadoop HDFS Architecture · Apache Parquet
- Apache Spark · Apache Iceberg · Delta Lake
- DuckDB · Elasticsearch Reference
- Apache Airflow · intégration GitHub–Zenodo

Cadres de la statistique officielle

- CEE-ONU — Modèle générique du processus de production statistique (GSBPM)
- CEE-ONU — Cadre de qualité pour les statistiques issues du big data
- SDMX · DDI · GSIM · W3C DCAT
- Comité d'experts des Nations unies sur le big data et la science des données pour la statistique officielle